CHAPTER 04

Software Construction

ACRONYMS

API	Application Programming Interface
ASIC	Application-Specific Integrated Circuit
BaaS	Backend As A Service
CI	Continuous Integration
COTS	Commercial Off-The-Shelf
CSS	Cascading Style Sheets
DSL	Domain-Specific Language
DSP	Digital Signal Processor
ESB	Enterprise Service Bus
FPGA	Field Programmable Gate Array
GPU	Graphic Processing Unit
GUI	Graphical User Interface
HTML5	Hypertext Markup Language Version 5
IDE	Integrated Development Environment
JEE	Jakarta Enterprise Edition
MDA	Model-Driven Architecture
NPM	Node Package Manager
OMG	Object Management Group
PIM	Platform Independent Model
POSIX	Portable Operating System Interface
PSM	Platform-Specific Model
SDK	Software Development Kit
TDD	Test-Driven Development
UML	Unified Modeling Language
WYSIWYG	What You See Is What You Get

INTRODUCTION

Software construction refers to the detailed creation and maintenance of software through coding, verification, unit testing, integration testing and debugging.

The software construction knowledge area (KA) is linked to all the other KAs, but it is most strongly linked to the Software Design and Software Testing KAs because the software construction process involves significant design and testing. The process uses the design output and provides an input to testing ("design" and "testing" in this case referring to the activities, not the KAs). Boundaries among design, construction and testing (if any) vary depending on the software life cycle processes used in a project.

Although some detailed design might be performed before construction, much design work is performed during construction. Thus, the Software Construction KA is closely linked to the Software Design KA.

Also, throughout construction, software engineers both unit-test and integration-test their work. Thus, the Software Construction KA is closely linked to the Software Testing KA as well.

The Software Construction KA is also related to configuration management, quality, project management and computing, and thus to the relevant KAs.

First, software construction typically produces the highest number of configuration items that need to be managed in a software project (e.g., source files, documentation, test cases). Thus, the Software Construction KA is closely linked to the Software Configuration Management KA.

Second, while quality is important in all the KAs, code is a software project's ultimate deliverable, and code is produced during construction. Thus, the Software Quality KA is closely linked to the Software Construction KA.

Third, while project management involves various software development tasks, software construction typically produces the most deliverables of a software project. Thus, the Software Construction KA is closely linked to the Software Engineering Management KA.

Fourth, since software construction requires knowledge of algorithms and coding practices, this KA is closely related to the Computing Foundations KA, which concerns the computer science foundations supporting software product design and construction.

BREAKDOWN OF TOPICS FOR SOFTWARE CONSTRUCTION

The breakdown of topics for the Software Architecture KA is shown in Figure 4-1.

1. Software Construction Fundamentals

Software construction fundamentals include the following:

- Minimizing complexity
- Anticipating and embracing change
- Constructing for verification
- Reusing assets
- Applying standards in construction

The first four concepts apply to design as well as to construction. The following sections define these concepts and describe how they apply to construction.

1.1. Minimizing Complexity [1, c2, c3, c7-9, c24, c27, c28, c3, 1, c32, c34]

All people have limited ability to hold complex structures and information in their working memories, especially over long periods. This greatly influences how people convey intent to computers and drives one of the key goals in software construction — to minimize complexity. The need to reduce complexity applies to essentially every aspect of software construction and is particularly critical to testing software constructions.

Several types of complexity can affect software construction. Tools can be used to manage different aspects of the complexity of software components and their construction. For example, cyclomatic complexity is a static analysis measure of how difficult code is to test and understand. The tool, developed by Thomas J. McCabe, Sr., in 1976, calculates the number of linearly independent paths through a program's source code. Ideally, there should be at least that number of test cases. Other examples are tools like Make, which can build an application, or integrated development environments (IDEs) for entering, editing and compiling code. These tools help manage the complexity of the construction process.

In software construction, reduced complexity is achieved by creating simple and readable code rather than clever code. This is accomplished by using standards (see section *3.1.5, Standards in Construction*), modular design (see section 3.1, Construction Design) and numerous other specific techniques (see section 3.3, Coding). Construction-focused quality techniques also support this (see section 3.6, Construction Quality).

1.2. Anticipating and Embracing Change [1-c3-c5, c24, c31, c32, c34, 2-c1, c3, c9, 3-c1]

Most software changes over time, and anticipating change drives many aspects of software construction; changes in the environments in which software operates also

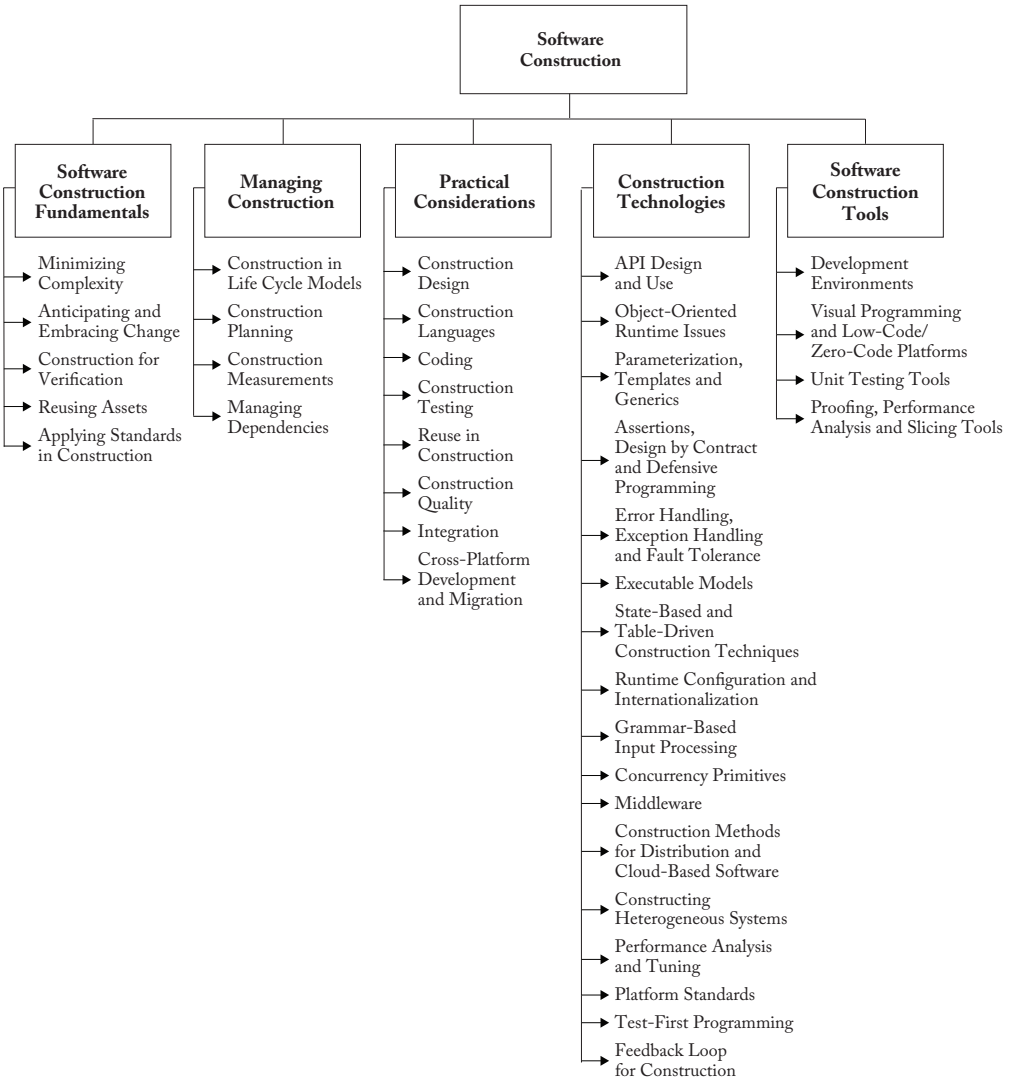

Figure 4.1. Breakdown of Topics for the Software Construction KA

affect software in diverse ways. Anticipating change helps software engineers build extensible software, enhancing a software product without disrupting the underlying structure. Anticipating change is supported by many specific techniques (see section 3.3, Coding).

Moreover, today's business environments require many organizations to deliver and deploy software more frequently, faster and more reliably. Anticipating specific, necessary changes can be difficult, so software engineers should be careful to build flexibility and adaptability into the software to incorporate changes with less difficulty. These software teams should embrace change by adopting agile development, practicing DevOps, and by adopting continuous delivery and deployment practices. Such practices align the software development process and management with an evolutionary environment.

1.3. Constructing for Verification [1-c8, c20-c23, c31, c34]

Constructing for verification builds software in such a way that faults can be readily found by the software engineers writing the software as well as by the testers and users during independent testing and operational activities. Specific techniques that support constructing for verification include following coding standards to support code reviews and unit testing, organizing code to support automated testing, restricting the use of complex or difficult-to-understand language structures, and recording software behaviors with logs.

1.4. Reusing Assets [2-c15]

Reuse means using existing assets to solve different problems. In software construction, typical assets that are reused include frameworks, libraries, modules, components, source code and commercial off-the-shelf (COTS) assets. Reuse has two closely related facets: *construction for reuse* and *construction with reuse*. The former means creating reusable software assets, whereas the latter means reusing software assets to construct a new solution. Reuse often transcends project boundaries, which means reused assets can be constructed in other projects or organizations.

1.5. Applying Standards in Construction [1-c4]

Applying external or internal development standards during construction helps achieve a project's efficiency, quality and cost objectives. Specifically, the choices of allowable programming language subsets and usage standards are important aids in achieving higher security.

Standards that directly affect construction issues include the following:

- Communication methods (e.g., standards for document formats and content)
- Programming languages (e.g., standards for languages like Java and C++)
- Coding standards (e.g., standards for naming conventions, layout and indentation)
- Exception handling policies (e.g., standards for the information included in exceptions and the way how exceptions are handled after catching)
- Platforms (e.g., interface standards for operating system calls)
- Tools (e.g., diagrammatic standards for notations like UML - Unified Modeling Language)

Use of external standards: Construction depends on external standards for construction languages, construction tools, technical interfaces and interactions between the Software Construction KA and other KAs. Standards come from numerous sources, including hardware and software interface specifications (e.g., Object Management Group (OMG)) and international organizations (e.g., the Institute of Electrical and Electronics Engineers (IEEE), the International Organization for Standardization (ISO)).

Use of internal standards: Standards may also be created on an organizational basis at the corporate level or for use on specific projects. These standards support coordinating group activities, minimizing complexity, anticipating change and constructing for verification.

2. Managing Construction

2.1. Construction in Life Cycle Models [1-c2, c3, c27, c29, 2-c3, c7, 3-c1]

Numerous models have been created to describe the development of software; some emphasize construction more than others.

Some models are more linear from the construction viewpoint, such as the waterfall and staged-delivery life cycle models. These models treat construction as an activity that occurs only after the completion of significant prerequisite work, including detailed requirements work, extensive design work and detailed planning. The more linear approaches emphasize the activities that precede construction

(requirements and design) and create more distinct separations between activities. In these models, construction's main emphasis might be coding.

Other models, such as evolutionary prototyping and agile development, are more iterative. These approaches treat construction as an activity that occurs concurrently with or overlaps other software development activities (including requirements, design and planning). These approaches mix design, coding and testing activities, and they often treat the combination of activities as construction (see the Software Engineering Management and Software Process KAs).

The practices of continuous delivery and deployment further mix coding, testing, delivery and deployment activities. In these practices, software updates made during construction activities are continuously delivered and deployed into the production environment. The whole process is fully automated by a deployment pipeline that consists of various testing and deployment activities.

Consequently, what is considered construction depends on the life cycle model used. In general, software construction is mostly coding and debugging, but it also involves construction planning, detailed design, unit testing, integration testing and other activities.

2.2. Construction Planning [1-c3, c4, c21, c27-c29]

The choice of construction method is a key aspect of the *construction planning* activity. This choice affects the extent to which construction prerequisites are performed, the order in which they are performed and the degree to which they should be completed before construction work begins.

The approach to construction affects the project team's ability to reduce complexity, anticipate change and construct for verification. Each objective may also be addressed at the process, requirements and design levels, but the choice of construction method will influence them.

Construction planning also defines the order in which components are created and integrated, the integration strategy (for example, phased or incremental integration), the software quality management processes, the allocation of task assignments to specific software engineers, and other tasks, according to the chosen method.

2.3. Construction Measurement [1-c25, c28]

Numerous construction activities and artifacts can be measured, including code developed, modified, reused, and destroyed; code complexity; code inspection statistics; fault-fix and fault-find rates; effort; and scheduling. These measurements can be useful for managing construction, ensuring quality during construction and improving the construction process, among other uses (see the Software Engineering Process KA for more on measurement).

2.4. Managing Dependencies [2-c25]

Software products often heavily rely on dependencies, including internal and external (commercial or open-source) dependencies, which allow developers to reuse common functionalities instead of reinventing the wheel and substantially improve developers' productivity. In addition, package managers (e.g., Maven in Java and NPM in JavaScript) are widely used to automate the process of installing, upgrading, configuring and removing dependencies.

The direct and indirect dependencies of software products constitute a dependency supply chain network. Any dependency in the supply chain network can introduce potential risk to software products and should be managed by developers or tools. Unnecessary dependencies should be avoided to improve build efficiency. License conflicts between dependencies and software products should be avoided to reduce legal risk. Propagation of dependencies' defects or vulnerabilities into software products should be avoided to improve the quality of software products. Regulations and monitoring

mechanisms should be developed to prevent developers from introducing untrusted external dependencies.

3. Practical Considerations

Construction is an activity in which the software engineer often has to deal with sometimes chaotic, changing and even conflicting real-world constraints. Because of real-world constraints, practical considerations drive construction more than some other KAs, and software engineering is perhaps most craft-like in the construction activities compared with other activities.

3.1. Construction Design [1-c3, c5, c24, 2-c7]

Some projects allocate considerable design activity to construction, whereas others allocate design to a phase explicitly focused on design. Regardless of the exact allocation, some detailed design work occurs at the construction level, and that design work is dictated by constraints imposed by the real-world problem the software addresses.

Just as construction workers building a physical structure must make small modifications for unanticipated gaps in the builder's plans, software construction workers must make small or large modifications to flesh out software design details during construction.

The details of the design activity at the construction level are essentially the same as described in the Software Design KA, but they are applied at a smaller scale to algorithms, data structures and interfaces.

3.2. Construction Languages [1-c4]

Construction languages include all forms of communication by which a human can specify an executable solution to a problem. Consequently, construction languages and their implementations (e.g., compilers) can affect software quality attributes such as performance, reliability and portability. As a result, they can seriously contribute to security vulnerabilities.

The simplest construction language is a *configuration language,* in which software engineers choose from a limited set of predefined options to create new or custom software installations. The text-based configuration files used in both the Windows and Unix operating systems are examples of this, and some program generators' menu-style selection lists constitute another example of a configuration language.

Toolkit languages are used to build applications from elements in toolkits (integrated sets of application-specific reusable parts); they are more complex than configuration languages. Toolkit languages may be explicitly defined as application programming languages, or the applications might be implied by a toolkit's set of interfaces.

Scripting languages are commonly used application programming languages. In some scripting languages, scripts are called *batch files* or *macros*.

Programming languages are the most flexible construction languages. They also contain the least amount of information about specific application areas and development processes. Therefore, they require the most training and skill to use effectively. The choice of programming language can greatly affect the likelihood of vulnerabilities being introduced during coding (e.g., unsafe use of C and C++ library functions is questionable from a security viewpoint).

Three general notations are used for programming languages:

- Linguistic (e.g., C/C++, Java)
- Formal (e.g., Event-B)
- Visual (e.g., MATLAB)

Linguistic notations are distinguished in particular by the use of textual strings to represent complex software constructions. The combination of textual strings in patterns may have a sentence-like syntax. Properly used, each string should have a strong semantic connotation providing an immediate intuitive understanding of what happens when the software construction is executed.

Formal notations rely less on intuitive, everyday meanings of words and text strings and more on definitions backed by precise, unambiguous and formal (or mathematical) definitions. Formal construction notations and methods are at the semantic base of most system programming notations, where accuracy, time behavior and testability are more important than ease of mapping into natural language. Formal constructions also use precisely defined ways of combining symbols that avoid the ambiguity of many natural language constructions.

Visual notations rely much less on the textual notations of linguistic and formal construction and more on direct visual interpretation and placement of visual entities that represent the underlying software. Visual construction is somewhat limited by the difficulty of making "complex" statements using only the arrangement of icons on a display. However, these icons can be powerful tools in cases where the primary programming task is to build and "adjust" a visual interface to a program, the detailed behavior of which has an underlying definition.

Nowadays, domain-specific languages (DSLs) are widely used to build domain-specific applications. Unlike a general-purpose programming language, such as C/C++ or Java, a DSL is designed for the application construction of a particular domain. Therefore, a DSL usually can be defined based on a higher level of abstraction of the target domain and can be optimized for a specific class of problems. Furthermore, A DSL usually can be expressed by visual notations defined by domain-specific concepts and rules.

3.3. Coding [1-c5-c19, c25-c26]

The following considerations apply to the software construction *coding* activity:

- Techniques for creating understandable source code, including naming conventions and source code layout
- Use of classes, enumerated types, variables, named constants and other similar entities
- Use of control structures
- Handling of error conditions — both anticipated and exceptional (e.g., input of bad data)
- Prevention of code-level security breaches (e.g., buffer overflows or array index bounds)
- Resource use through use of exclusion mechanisms and discipline in accessing serially reusable resources, including threads and database locks
- Source code organization into statements, routines, classes, packages or other structures
- Code documentation
- Code tuning

3.4. Construction Testing [1-c22, c23, 2-c8]

Construction involves two forms of testing, which are often performed by the software engineer who wrote the code: unit testing and integration testing.

Construction testing aims to reduce the gap between when faults are inserted into the code and when those faults are detected, thereby reducing the cost incurred to fix them. In some instances, test cases are written after the code has been written. In other instances, test cases might be created before code is written.

Construction testing typically involves a subset of the various types of testing, described in the Software Testing KA. For instance, construction testing does not typically include system testing, alpha testing, beta testing, stress testing, configuration testing, usability testing, or other more specialized testing.

Two standards have been published on construction testing: IEEE Standard 829-2008, "IEEE Standard for Software Test Documentation," and "IEEE Standard for Software Unit Testing."

See sections 2.1.1 and 2.1.2 in the Software Testing KA for more specialized reference material.

3.5. Reuse in Construction [2-c15, c16]

Reuse in construction includes both construction for reuse and construction with reuse.

Construction for reuse creates software with the potential to be reused in the future for the present project or for other projects with a broad-based, multisystem perspective. Construction for reuse is usually based on variability analysis and design. To avoid the problem of code clones, developers should encapsulate reusable code fragments into well-structured libraries or components.

The tasks related to software construction for reuse during coding and testing are as follows:

- Variability implementation with mechanisms such as parameterization, conditional compilation and design patterns
- Variability encapsulation to make the software assets easy to configure and customize
- Testing the variability provided by the reusable software assets
- Description and publication of reusable software assets

Construction with reuse means creating new software by reusing existing software assets. The most popular reuse method is to reuse code from the libraries provided by the language, platform, tools or an organizational repository. Aside from these, many applications developed today use third-party open-source libraries. In addition, reused and off-the-shelf software often have the same (or better) quality requirements as newly developed software (e.g., security level requirements).

The tasks related to software construction with reuse during coding and testing are as follows:

- Selecting reusable units, databases, test procedures or test data
- Evaluating code or test reusability
- Integrating reusable software assets into the current software
- Reporting reuse information on new code, test procedures or test data

The forms of reusable software assets are not limited to software artifacts that must be locally integrated. Nowadays, cloud services that provide various services through online interfaces such as RESTful application programming interfaces (APIs) are widely used in applications. In the new cloud service model BaaS (backend as a service), applications delegate their backend implementations to cloud service providers — for example, utilities such as authentication, messaging and storage are usually provided by cloud providers.

Reuse is best practiced systematically, according to a well-defined, repeatable process. Systematic reuse can enable significant software productivity, quality and cost improvements. Systematic reuse is supported by methodologies such as software product line engineering and various software frameworks and platforms. Widely used frameworks such as Spring provide reusable infrastructures for enterprise applications so software teams can focus on application-specific business logic. Commercial platforms provide various reusable frameworks, libraries, components and tools to support application development to build their ecosystems.

3.6. Construction Quality [1-c8, c20-c25, 2-c8, c24]

In addition to faults occurring during requirements and design activities, faults introduced during construction can cause serious quality problems (e.g., security vulnerabilities). These include not only faults in security functionality but also faults elsewhere that allow bypassing of the security functionality or create other security weaknesses or violations.

Numerous techniques exist to ensure the quality of code as it is constructed. The primary techniques used to ensure construction quality are:

- Unit testing and integration testing (see section 3.4, Construction Testing)

- Test-first development (see section 6.1.2 in the Software Testing KA)
- Use of assertions and defensive programming
- Debugging
- Inspections
- Technical reviews, including security-oriented reviews (see section 2.3 in the Software Quality KA)
- Static analysis (see section 2.2.1 of the Software Quality KA)

The specific technique or techniques selected depend on the software constructed and on the skill set of the software engineers performing the construction activities. Programmers should know good practices and common vulnerabilities (e.g., from widely recognized lists about common vulnerabilities). Automated static code analysis for security weaknesses is available for several common programming languages.

Construction quality activities are differentiated from other quality activities by their focus. These activities focus on artifacts that are closely related to code — such as detailed design — as opposed to other artifacts that are less directly connected to the code, such as requirements, high-level designs and plans.

3.7. Integration [1-c29, 2-c8, 3-c11]

During construction, a key activity is integrating individually constructed routines, classes, components and subsystems into a single system. In addition, a particular software system may need to be integrated with other software or hardware systems.

Concerns related to construction integration include planning the sequence in which components are integrated, identifying what hardware is needed, creating scaffolding to support interim versions of the software, determining the degree of testing and quality work performed on components before they are integrated, and determining points in the project at which interim versions of the software are tested.

Programs can be integrated by means of either the phased or the incremental approach. Phased integration, also called *big bang* integration, entails delaying the integration of component software parts until all parts intended for release in a version are complete. Incremental integration is thought to offer many advantages over the traditional phased integration (e.g., easier error location, improved progress monitoring, earlier product delivery and improved customer relations). In incremental integration, the developers write and test a program in small pieces and then combine the pieces one at a time. Additional test infrastructure, including, for example, stubs, drivers and mock objects, is usually needed to enable incremental integration. In addition, by building and integrating one unit at a time (e.g., a class or component), the construction process can provide early feedback to developers and customers.

Today, continuous integration (CI) has been widely adopted in practice. A software team using CI integrates its work frequently, leading to multiple integrations per day. CI is usually automated by a pipeline that builds and tests each integration to detect errors and provide fast feedback.

3.8. Cross-Platform Development and Migration [4-c]

Some applications, such as mobile applications, heavily rely on specific platforms (e.g., Apple, Android), which usually include operating systems, development frameworks and APIs. To support multiple platforms, the developers need to develop and build an application separately for each target platform using the corresponding program language and software development kit (SDK). However, multi-platform development in this way requires more time and cost and might cause different user experiences between different implementations.

Cross-platform development allows the developers to develop an application using a universal language and export it to various platforms. This usually can be done in two

ways for mobile applications. One way is to generate native applications using tools that can compile the universal language into platform-specific formats. The other is to develop hybrid applications that combine web applications developed using languages like hypertext markup language version 5 (HTML5) and cascading style sheets (CSS) and native containers or wrappers for various operations systems.

For applications that are not developed in this way, developers may consider migrating the applications from one platform to another. The migration usually involves translation of different programming languages and platform-specific APIs and can be partially automated by tools.

4. Construction Technologies

4.1. API Design and Use [5-c7]

An *API* is a set of signatures that are exported and available to the users of a library or a framework to write their applications. Besides signatures, an API should always include statements about the program's effects and/or behaviors (i.e., its semantics).

API design should make the API easy to learn and memorize, lead to readable code, be difficult to misuse, be easy to extend, be complete, and maintain backward compatibility. As the APIs usually outlast their implementations for a widely used library or framework, an API should be straightforward and stable, to facilitate client application development and maintenance.

API use involves selecting, learning, testing, integrating and possibly extending APIs provided by a library or framework (see section 3.5, Reuse in Construction).

For online interfaces such as RESTful APIs, open standards such as OpenAPI play an important role. OpenAPI defines a standard, language-agnostic interface to HTTP APIs and supports the automatic generation of server-side and client-side code, covering popular languages such as Java, JavaScript, Python, etc. At the same time, the API-first approach has been widely used, which emphasizes designing and building the APIs of an application first. In practice, the API-first approach is usually accomplished by using an API description language to establish a contract for how the API is supposed to behave.

4.2. Object-Oriented Runtime Issues [1-c6, c7]

Object-oriented languages support runtime mechanisms, including polymorphism and reflection. These runtime mechanisms increase the flexibility and adaptability of object-oriented programs.

Polymorphism is a language's ability to support general operations without knowing until runtime what kind of concrete objects the software will include. Because the program does not know the types of the objects in advance, the exact behavior is determined at runtime (called *dynamic binding*).

Reflection is a program's ability to observe and modify its structure and behavior at runtime. For example, reflection allows inspection of classes, interfaces, fields and methods at runtime without knowing their names at compile time. It also allows instantiation of new objects at runtime and invocation of methods using parameterized class and method names.

4.3. Parameterization, Templates, and Generics [6-c1]

Parameterized types, also known as *generics* (Ada, Java, Eiffel) and *templates* (C++), enable a type or class definition without specifying all the other types used. The unspecified types are supplied as parameters at the point of use. Parameterized types provide a third way (besides class inheritance and object composition) to compose behaviors in object-oriented software.

4.4. Assertions, Design by Contract, and Defensive Programming [1-c8, c9]

An *assertion* is an executable predicate placed in a program — usually a routine or macro — that performs runtime checks of the program.

Assertions are especially useful in high-reliability programs. They enable programmers to more quickly flush out mismatched interface assumptions, errors that creep in when code is modified, and other problems. Assertions are typically compiled into the code at development time and are later compiled out of the code so they don't degrade the performance.

Design by contract is a development approach in which preconditions and postconditions are included for each routine. When preconditions and postconditions are used, each routine or class is said to form a contract with the rest of the program. A contract precisely specifies the semantics of a routine and thus helps clarify its behavior. Design by contract is thought to improve the quality of software construction.

Defensive programming means to protect a routine from being broken by invalid inputs. Common ways to handle invalid inputs include checking the values of all the input parameters and deciding how to handle bad inputs. Assertions are often used in defensive programming to check input values.

4.5. Error Handling, Exception Handling, and Fault Tolerance [1-c8, c9]

How *errors* are handled affects software's ability to meet requirements related to correctness, robustness and other nonfunctional attributes. Assertions are sometimes used to check for errors. Other error-handling techniques — such as returning a neutral value, substituting the next piece of valid data, logging a warning message, returning an error code or shutting down the software — are also used.

Exceptions are used to detect and process errors or exceptional events. The basic structure of an exception is as follows: A routine uses *throw* to throw a detected exception, and an exception-handling block will *catch* the exception in a *try-catch* block. The try-catch block may process the erroneous condition or return control to the calling routine. Exception-handling policies should be carefully designed following common principles, such as including in the exception message all information that led to the exception, avoiding empty catch blocks, knowing the exceptions the library code throws, perhaps building a centralized exception reporter, and standardizing the program's use of exceptions.

Fault tolerance is a collection of techniques that increase software reliability by detecting errors and then recovering from them or containing their effects if recovery is not possible. The most common fault tolerance strategies include backing up and retrying, using auxiliary code and voting algorithms, and replacing an erroneous value with a phony value that will have a benign effect.

4.6. Executable Models [7]

Executable models abstract away the details of specific programming languages and decisions about the software's organization. Different from traditional software models, a specification built in an executable modeling language like xUML (executable UML) can be deployed in various software environments without change. Furthermore, an executable-model compiler (transformer) can turn an executable model into an implementation using a set of decisions about the target hardware and software environment. Thus, constructing executable models is a way of constructing executable software.

Executable models are one foundation supporting the model-driven architecture (MDA) initiative of the OMG. An executable model is a way to specify a platform-independent model (PIM); a PIM is a model of a solution to a problem that does not rely on any implementation technologies. Then a platform-specific model (PSM), which is a model that contains the details of the implementation, can be produced by weaving together the PIM and the platform on which it relies.

4.7. State-Based and Table-Driven Construction Techniques [1-c18]

State-based programming, or *automata-based programming*, is a programming technology

that uses finite-state machines to describe program behaviors. A state machine's transition graphs are used in all stages of software development (specification, implementation, debugging and documentation). The main idea is to construct computer programs in the same way technological processes are automated. State-based programming is usually combined with object-oriented programming, forming a new composite approach called *state-based, object-oriented programming.*

A *table-driven* method is a schema that uses tables to display information rather than convey information with logic statements (such as *if* and *case*). When used in appropriate circumstances, table-driven code is simpler than complicated logic and easier to modify. When using table-driven methods, the programmer addresses two issues: what information to store in the table or tables and how to efficiently access information in the table.

4.8. Runtime Configuration and Internationalization [1-c3, c10]

To achieve more flexibility, a program is often constructed to support its variables' late binding time. For example, *runtime configuration* binds variable values and program settings when the program is running, usually by updating and reading configuration files in a just-in-time mode.

Internationalization is the technical activity of preparing a program, usually interactive software, to support multiple locales. The corresponding activity, *localization,* modifies a program to support a specific local language. Interactive software may contain dozens or hundreds of prompts, status displays, help messages, error messages and so on. The design and construction processes should accommodate string and character set issues, including which character set is used, what kinds of strings are used, how to maintain the strings without changing the code and how to translate the strings into different languages with minimal impact on the processing code and the user interface.

4.9. Grammar-Based Input Processing [1, 8]

Grammar-based input processing involves syntax analysis, or *parsing*, of the input token stream. It involves the creation of a data structure (called a *parse tree* or *syntax tree*) representing the input data. The inorder traversal of the parse tree usually gives the expression just parsed. Next, the parser checks the symbol table for programmer-defined variables that populate the tree. After building the parse tree, the program uses it as an input to the computational processes.

4.10. Concurrency Primitives [9-c6]

A *synchronization primitive* is a programming abstraction provided by a programming language or the operating system that facilitates concurrency and synchronization. Well-known concurrency primitives include semaphores, monitors and mutexes.

A *semaphore* is a protected variable or abstract data type that provides a simple but useful abstraction for controlling access to a common resource by multiple processes or threads in a concurrent programming environment.

A *monitor* is an abstract data type that presents a set of programmer-defined operations executed with mutual exclusion. A monitor contains the declaration of shared variables and procedures or functions that operate on those variables. The monitor construct ensures that only one process at a time is active in the monitor.

A *mutex* (mutual exclusion) is a synchronization primitive that grants exclusive access to a shared resource by only one process or thread at a time.

4.11. Middleware [5-c1, 8-c8]

Middleware is a broad classification for software that provides services above the operating system layer yet below the application program layer. Middleware can provide runtime containers for software components to provide message passing, persistence and a transparent

location across a network. Middleware can be viewed as a connector between the components using the middleware. Modern message-oriented middleware usually provides an enterprise service bus (ESB) that supports service-oriented interaction and communication among multiple software applications.

4.12. Construction Methods for Distributed and Cloud-Based Software [2-c17, c18, 9-c2]

A *distributed system* is a collection of physically separate, possibly heterogeneous computer systems networked to provide the users with access to the resources the system maintains. The construction of distributed software is distinguished from traditional software construction by issues such as parallelism, communication and fault tolerance.

Distributed programming typically falls into several basic architectural categories: client-server, three-tier architecture, n-tier architecture, distributed objects, loose coupling or tight coupling (see section 5.6 in the Computing Foundations KA and section 2.2 in the Software Architecture KA).

Nowadays, more applications are migrated to the cloud. *Cloud-based software* often adopts microservice architecture and container-based deployment. In addition to traditional distributed software issues, cloud-based software developers also need to consider cloud infrastructure issues such as use of an API gateway, service registration and discovery.

Distributed systems based on n-tier/service-oriented architectures usually rely on ACID distributed transactions for the implementation of transactions involving multiple distributed components. In contrast, cloud-based microservices cannot enforce distributed transactions consistency, and use some form of SAGA-based eventual consistency, initially intended for long-running transactions.

4.13. Constructing Heterogeneous Systems [8-c9]

Heterogeneous systems consist of various specialized computational units of different types, such as Graphic Processing Units (GPUs) and Digital Signal Processors (DSPs), microcontrollers and peripheral processors. These computational units are independently controlled and communicate with one another. Embedded systems are typically heterogeneous systems.

The design of heterogeneous systems may require combining several specification languages to design different system parts (hardware/software codesign). The key issues include multilanguage validation, co-simulation and interfacing.

During the hardware/software codesign, software and virtual hardware development proceed concurrently through stepwise decomposition. The hardware part is usually simulated in field programmable gate arrays (FPGAs) or application-specific integrated circuits (ASICs). The software part is translated into a low-level programming language.

4.14. Performance Analysis and Tuning [1-c25, c26]

Code efficiency — determined by architecture, detailed design decisions, and data structure and algorithm selection — influences execution speed and size. *Performance analysis* investigates a program's behavior using information gathered as the program executes to identify possible hot spots in the program to be improved.

Code *tuning*, which improves performance at the code level, modifies code to make it run more efficiently. Code tuning usually involves only small changes that affect a single class, a single routine or, more commonly, a few lines of code. A rich set of code tuning techniques is available, including those for tuning logic expressions, loops, data transformations, expressions and routines. Using a low-level language is another common technique for improving hot spots in a program.

4.15. Platform Standards [4-c, 8-c10, 9-c1]

Platform standards enable programmers to develop portable applications that can be executed in compatible environments without

changes. Platform standards usually involve standard services and APIs that compatible platform implementations must use. Typical examples of platform standards are Jakarta Enterprise Edition (JEE); the portable operating system interface (POSIX) standard for operating systems, which represents a set of standards implemented primarily for Unix-based operating systems; and HTML5, which defines the standards for developing web applications that can run on different environments (e.g., Apple iOS, Android).

4.16. Test-First Programming [1-c22, 2-c8]

Test-first programming (also known as TDD - Test-Driven Development) is a popular development style in which test cases are written before any code. These test cases, when applied to the current code base, will fail. Code is then written that will allow the test cases to pass. At that time, the new code and associated parts of the project can be refactored and optimized. Test-first programming can usually detect defects earlier and correct them more easily than traditional programming styles. Furthermore, writing test cases first forces programmers to think about requirements and design before coding, thus exposing requirements and design problems sooner.

4.17. Feedback Loop for Construction [3-c3, c16]

Early and continuous feedback for the construction activity is one of the most important advantages of agile development and DevOps. Agile development provides early feedback for construction through frequent iterations in the development process. DevOps provides even faster feedback from the operation, allowing the developers to learn how well their code performs in production environments. This fast feedback is achieved through techniques and practices in the DevOps pipeline, such as automated building and testing, canary release, and A/B testing.

5. Software Construction Tools

5.1. Development Environments [1-c30]

A *development environment*, or *integrated development environment* (IDE), provides comprehensive facilities to programmers for software construction by integrating a set of development tools. The programmers' choice of development environment can affect software construction efficiency and quality.

Besides basic code editing functions, modern IDEs often offer other features, like compilation and error detection within the editor, integration with source code control, build/test/debugging tools, condensed or outline views of programs, automated code transforms, and support for refactoring.

Nowadays, cloud-based development environments are available in public or private cloud services. These environments can provide all the features of modern IDEs and even more (e.g., containerized building and deployment), powered by the cloud.

Moreover, modern IDEs are often equipped with AI-assisted programming which is boosted by the recent advances in Large Language Models (LLMs). With the support a programmer can define a function in pseudocode comments or outline its implementation as a prompt for an LLM to generate or complete the code. The programmer lets the LLM complete many of the details, but still reviews the generated code and integrates it into their project.

5.2. Visual Programming and Low-Code/Zero-Code Platforms [1-c30]

Visual programming allows users to create programs by manipulating visual program elements graphically. As a visual programming tool, a GUI (graphical user interface) builder enables the developer to create and maintain GUIs in a WYSIWYG (what you see is what you get) mode. A GUI builder usually includes a visual editor that enables the developer to design forms and windows and manage the layout of the widgets with drag,

drop and parameter setting features. Some GUI builders can automatically generate the source code corresponding to the visual GUI design. Because GUI applications usually follow the event-driven style (in which events and event handling determine the program flow), GUI builder tools usually provide code generation assistants, which automate the most repetitive tasks required for event handling. The supporting code connects widgets with the outgoing and incoming events that trigger the functions providing the application logic. Some modern IDEs provide integrated GUI builders or GUI builder plug-ins. There are also many stand-alone GUI builders.

Visual programming and other rapid application development tools have evolved into *low-code/zero-code platforms*. These platforms allow developers to build complete applications visually through a drag-and-drop interface and with minimal hand-coding. They are usually based on the principles of model-driven design, visual programming and code generation. The difference between low-code development and zero-code development lies in hand-coding; the former requires a little hand-coding, whereas the latter requires practically none.

5.3. Unit Testing Tools [1-c22, 2-c8]

Unit testing verifies the functioning of software modules in isolation from other separately testable software elements (for example, classes, routines, components). Unit testing is often automated. Developers can use unit testing tools and frameworks to extend and create an automated testing environment. For example, the developer can code criteria into the test with unit testing tools and frameworks to verify the unit's correctness under various data sets. Each test is implemented as an object, and a test runner runs the tests. Failed test cases are automatically flagged and reported during the test execution.

5.4. Profiling, Performance Analysis, and Slicing Tools [1-c25, c26]

Performance analysis tools are usually used to support code tuning. The most common performance analysis tools are *profiling tools*. An execution profiling tool monitors the code while it runs and records how often each statement is executed or how much time the program spends on each statement or execution path. Profiling the code while it runs gives insight into how the program works, where the hot spots are and where the developers should focus the code tuning efforts.

Program slicing involves computing the set of program statements (i.e., the program slice) that might affect the values of specified variables at some point of interest, which is called a slicing criterion. Program slicing can be used for locating error sources, program understanding and optimization analysis. Program slicing tools compute program slices for various programming languages using static or dynamic analysis methods.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	McConnell, 2004 [1]	Sommerville, 2016 [2]	Kim et al., 2021 [3]	Heitkötter et al., 2013 [4]	Clements et al., 2010 [5]	Gamma et al. 1994 [6]	Mellor and Balcer, 2002 [7]	Null and Lobur, 2018 [8]	Silberschatz et al., 2008 [9]
1. Software Construction Fundamentals									

1.1. Minimizing Complexity	c2, c3, c7-c9, c24, c27, c28, c31, c32, c34								
1.2. Anticipating and Embracing Change	c3-c5, c24, c31, c32, c34	c1, c3, c9	c1						
1.3. Constructing for Verification	c8, c20-c23, c31, c34								
1.4. Reuse		c15							
1.5. Standards in Construction	c4								
2. Managing Construction									
2.1. Construction in Life Cycle Models	c2, c3, c27, c29	c3, c7	c1						
2.2. Construction Planning	c3, c4, c21, c27-c29								
2.3. Construction Measurement	c25, c28								
2.4. Managing Dependencies		c25							
3. Practical Considerations									
3.1. Construction Design	c3, c5, c24	c7							
3.2. Construction Languages	c4								
3.3. Coding	c5-c19, c25-c26								
3.4. Construction Testing	c22, c23	c8							
3.5. Reuse in Construction		c15, c16							
3.6. Construction Quality	c8, c20-c25	c8, c24							
3.7. Integration	c29	c8	c11						
3.8. Cross-Platform Development and Migration				c					
4. Construction Technologies									
4.1. API Design and Use					c7				
4.2. Object-Oriented Runtime Issues	c6, c7								

4.3. Parameterization, Templates and Generics						c1			
4.4. Assertions, Design by Contract and Defensive Programming	c8, c9								
4.5. Error Handling, Exception Handling and Fault Tolerance	c3, c8								
4.6. Executable Models									
4.7. State-Based and Table-Driven Construction Techniques	c18								
4.8. Runtime Configuration and Internationalization	c3, c10								
4.9. Grammar-Based Input Processing	c5							c8	
4.10. Concurrency Primitives									c6
4.11. Middleware					c1			c8	
4.12. Construction Methods for Distributed and Cloud-Based Software		c17, c18							c2
4.13. Constructing Heterogeneous Systems								c9	
4.14. Performance Analysis and Tuning	c25, c26								
4.15. Platform Standards				c				c10	c1
4.16. Test-First Programming	c22	c8							
4.17. Feedback Loop for Construction			c3, c16						
5. Software Construction Tools									
5.1. Development Environments	c30								
5.2. Visual Programming and Low-Code/Zero-Code Platforms	c30								
5.3. Unit Testing Tools	c22	c8							
5.4. Profiling, Performance Analysis and Slicing Tools	c25, c26								

FURTHER READINGS

IEEE Std. 1517-2010: IEEE Standard for Information Technology — Software Life Cycle Processes — Reuse Processes, IEEE, 1999 [8].

This standard specifies the processes, activities, and tasks to be applied during each phase of the software life cycle to enable a software product to be constructed from reusable assets. It covers the concept of reuse-based development and the processes of construction for reuse and construction with reuse.

ISO/IEC 12207:2008: Information Technology — Software Life Cycle Processes, ISO/IEC, 2008 [9].

This standard defines a series of software development processes, including software construction process, software integration process, and software reuse process.

Martin Fowler, Kent Beck. Refactoring: Improving the Design of Existing Code (2nd Edition), Addison-Wesley Signature Series (Fowler).

Robert C. Martin. Clean Code: A Handbook of Agile Software Craftsmanship, Pearson Education, Inc.

REFERENCES

[1] S. McConnell, *Code Complete*, 2nd edition, Redmond, WA: Microsoft Press, 2004.

[2] I. Sommerville, *Software Engineering*, 10th edition, Addison-Wesley, 2016.

[3] G. Kim et al., *The DevOps Handbook: How to Create World-Class Agility, Reliability & Security in Technology Organizations*, 2nd edition, IT Revolution, 2021.

[4] H. Heitkötter, S. Hanschke, and T.A. Majchrzak, Evaluating Cross-Platform Development Approaches for Mobile Applications, 2013, in Cordeiro, J., Krempels, K.H. (eds.), *Web Information Systems and Technologies*. WEBIST 2012. Lecture Notes in Business Information Processing, vol. 140, Springer, Berlin, Heidelberg.

[5] P. Clements et al., *Documenting Software Architectures: Views and Beyond*, 2nd edition, Boston: Pearson Education, 2010.

[6] E. Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, 1st edition, Reading, MA: Addison-Wesley Professional, 1994.

[7] S.J. Mellor and M.J. Balcer, *Executable UML: A Foundation for Model-Driven Architecture*, 1st edition, Boston: Addison-Wesley, 2002.

[8*] L. Null and J. Lobur, *The Essentials of Computer Organization and Architecture*, 5th ed., Jones and Bartlett Publishers, 2018.

[9] A. Silberschatz et al., *Operating System Concepts*, 8th edition, Hoboken, NJ: Wiley, 2008.